DENSITY-AWARE HIERARCHICAL FRAMEWORK FOR POINT SET REPRESENTATIONS

Anonymous authors
Paper under double-blind review

ABSTRACT

This paper introduces a novel hierarchical framework for representing point sets using density-based clustering and relative geometric encoding. By integrating DBSCAN for adaptive partitioning and recursive centroid computation, our approach addresses limitations in existing methods such as sensitivity to sampling density and fixed neighborhood parameters. Evaluations on synthetic datasets demonstrate scalability and robustness across dimensions, paving the way for applications in 3D point cloud analysis.

1 INTRODUCTION

The rapid growth of applications involving high-dimensional point sets, ranging from 3D vision and robotics to molecular modeling and geospatial analysis, has driven a demand for effective representation learning methods. Unlike structured data such as images or sequences, point sets are inherently unordered and often irregularly distributed, which poses fundamental challenges for deep learning models. A central issue lies in designing representations that are both permutation-invariant and capable of capturing local geometric relationships. Over the past few years, point cloud learning has made remarkable progress, yet the problem of efficiently representing complex point sets while preserving their multi-scale structure remains far from solved.

Early methods addressed this challenge by directly processing point coordinates through architectures such as PointNet **?**, which established the principle of applying shared multilayer perceptrons to individual points followed by global aggregation. While effective in achieving permutation invariance, this approach largely ignored local structures and was therefore limited in its ability to capture fine-grained geometric patterns. Subsequent improvements, including PointNet++ **?** and graph-based networks, introduced mechanisms to model local neighborhoods. Although these developments significantly advanced the field, they often relied on heuristic parameters such as neighborhood radius or nearest-neighbor counts, which can be highly sensitive to variations in sampling density. As a result, these models may not generalize robustly across heterogeneous datasets or across point sets of different scales.

In parallel, attention-based and transformer-inspired approaches have expanded the expressive power of point set models. By adapting self-attention mechanisms and incorporating positional encodings, these methods are able to capture both global context and fine local details, and when combined with pretraining strategies, they have demonstrated strong generalization across tasks. At the same time, spatial partitioning frameworks such as octrees have offered efficient data structures that allow deep models to scale to larger point sets. Despite these successes, however, these approaches often carry structural assumptions that do not naturally align with the intrinsic density distribution of the data. In practice, this can lead to inefficiencies in representing sparsely populated regions or in handling highly irregular geometries.

These limitations suggest that an alternative perspective may be valuable. Instead of relying on fixed neighborhood definitions, global attention mechanisms, or axis-aligned partitions, one could seek representations that are inherently density-aware. Density-based clustering methods such as DBSCAN **?** provide a natural foundation in this regard: they identify clusters of points based on local density without requiring prior knowledge of the number of clusters, while also distinguishing outliers as noise. By leveraging such clustering as a core operator, it becomes possible to construct hierarchical representations where clusters are recursively summarized by their centers and local

structures are encoded in terms of relative distances and directions to these centers. This representation naturally forms a tree-like or forest-like hierarchy that can reduce redundancy, scale to large datasets, and preserve meaningful geometric relations.

Motivated by these observations, our work investigates a framework that integrates density-based clustering into the design of point set representations. By focusing on relative encoding with respect to cluster centers, we aim to develop a representation that is compact yet expressive, capable of scaling across different point set dimensions while maintaining adaptability to variations in density and distribution. While the details of this framework are elaborated in subsequent sections, the present discussion highlights the conceptual gap that our study seeks to address: bridging advances in point cloud learning with the underutilized potential of density-aware clustering for building multi-scale, hierarchical representations.

2 Related Work

Early progress in point cloud learning was driven by hierarchical feature learning architectures that extended the pioneering work of PointNet **?**. PointNet++ **?** introduced a recursive framework that builds on local neighborhoods, capturing fine-grained geometric information by applying PointNet modules at multiple scales. This approach addressed one of the key limitations of PointNet, namely its inability to model local structures, and demonstrated robustness to non-uniform sampling densities. Dynamic Graph CNN (DGCNN) **?** further advanced this line of work by proposing the EdgeConv operator, which dynamically updates local graphs during training. Unlike static neighborhood definitions, DGCNN recalculates point-to-point relationships at each layer, thereby allowing the network to adaptively refine its understanding of local geometric structures. These contributions marked an important step toward models that can flexibly capture both global and local features in point sets.

Parallel to these developments, attention mechanisms and transformer-based architectures have gained significant traction. Point Transformer **?** adapted the self-attention paradigm, originally designed for sequential data, to unordered point sets. By incorporating carefully designed positional encodings, it allowed the model to account for both global context and local geometry simultaneously, achieving state-of-the-art results on challenging benchmarks such as semantic and part segmentation. Building on the success of self-supervised learning in natural language processing, Point-BERT **?** proposed a masked point modeling strategy in which local point patches are discretized into tokens through a variational autoencoder and then reconstructed using a transformer. This pretraining scheme significantly improved transferability and downstream performance, illustrating the potential of large-scale self-supervised learning for 3D data. Later refinements, such as McP-BERT, reduced the ambiguity in patch tokenization by allowing multiple token assignments, thereby providing more stable and expressive pretrained representations. Collectively, these transformer-based methods highlighted the versatility of attention in handling irregular, unordered inputs while also underscoring the challenges of balancing computational cost with expressive power.

Another influential line of research has explored spatial partitioning through hierarchical or tree-structured representations. OctNet **?** represented 3D data with unbalanced octrees, enabling efficient allocation of computation to occupied regions while dramatically reducing memory usage compared to dense voxel grids. This framework allowed convolutional neural networks to process high-resolution 3D shapes that would otherwise be computationally prohibitive. Similarly, O-CNN **?** extended octree-based methods by building deeper convolutional architectures on top of octree representations, offering scalable solutions for large-scale 3D analysis. These methods showcased the potential of tree-based structures to bridge the gap between voxel-based and point-based representations, achieving higher resolution without incurring prohibitive resource requirements. However, their reliance on axis-aligned splits introduced a strong inductive bias that may not reflect the intrinsic geometry of complex or irregular point distributions.

Despite these advances, current approaches face inherent challenges. Hierarchical neighborhood-based methods such as PointNet++ and DGCNN depend on hyperparameters like radius thresholds or k-nearest neighbor counts, which are highly sensitive to variations in density and scale. This makes them less robust in heterogeneous or noisy environments. Transformer-based architectures mitigate some of these issues but often rely on absolute positional encodings or computationally expensive attention mechanisms, which limit their scalability to very large point sets. Pretraining

methods like Point-BERT have opened promising directions, yet the complexity of tokenization and reconstruction pipelines introduces additional overhead, and the learned representations are not always density-aware. Similarly, tree-structured approaches provide efficient scaling but impose rigid spatial priors that may fail to capture the nuanced density patterns present in real-world point clouds, particularly in high-dimensional or irregular spaces.

Taken together, the trajectory of research in point cloud representation illustrates steady progress from per-point encoding toward increasingly sophisticated frameworks that exploit hierarchical grouping, attention mechanisms, pretraining strategies, and spatial data structures. Nevertheless, the use of density-based clustering as a fundamental operator remains relatively underexplored. Algorithms such as DBSCAN **?** are occasionally used as preprocessing tools for segmentation or outlier detection, but they have not been fully leveraged to construct multi-scale, density-aware representations. This gap motivates our approach, which explores how density-driven clustering combined with relative geometric encoding can offer a more adaptive and scalable representation of point sets across diverse dimensions.

3 METHODOLOGY

We propose a hierarchical framework for point set representation that leverages density-based clustering and relative encoding of local geometry. Unlike conventional methods that rely on fixed neighborhood parameters or axis-aligned partitions, our approach adapts directly to the intrinsic distribution of data. This section details the design of each component in the framework, explains parameter selection strategies, and situates our design choices in relation to prior work.

3.1 DENSITY-BASED PARTITIONING

Given a point set

$$X = \{x_1, x_2, \ldots, x_n\}, \quad x_i \in \mathbb{R}^d,$$

the first step is to partition the data into clusters using the DBSCAN algorithm **?**. We adopt DBSCAN because of its well-documented ability to discover clusters of arbitrary shape without requiring the number of clusters as a prior input. In contrast, partitioning methods such as k-means **?** assume convex cluster boundaries and mandate pre-specification of cluster counts, which often leads to brittle behavior when data distribution is heterogeneous. DBSCAN, by relying on density connectivity rather than distance to a centroid, naturally accommodates irregular geometries, variable densities, and noisy points—characteristics that are common in real-world point clouds. For each point x_i, DBSCAN defines its neighborhood as

$$\mathcal{N}_\varepsilon(x_i) = \{x_j \in X \mid \|x_j - x_i\|_2 \leq \varepsilon\},$$

where $\varepsilon > 0$ denotes the radius parameter. A point is labeled as a *core point* if $|\mathcal{N}_\varepsilon(x_i)| \geq$ MinPts, where MinPts is the minimum number of points required to form a dense region. Clusters are formed by connecting density-reachable core points, while points not assigned to any cluster are designated as noise. The parameters ε and MinPts play a crucial role. Prior literature has shown that poor parameter selection can fragment natural clusters or merge distinct ones. To address this, we assume the use of k-distance plots, a technique introduced in density-based clustering research that ranks pairwise distances and identifies an inflection point corresponding to an appropriate ε. Similarly, MinPts is often recommended to be set slightly higher than the dimensionality of the dataset, typically between $d+1$ and $2d$, which balances sensitivity to local density with robustness to noise. These heuristics, validated in prior studies of density-based clustering, ensure that DBSCAN adapts to the intrinsic characteristics of the dataset rather than relying on arbitrary scale assumptions.

3.2 CLUSTER CENTER COMPUTATION

After partitioning, suppose the dataset is divided into

$$\mathcal{C} = \{C_1, C_2, \ldots, C_K\}, \quad C_j \subseteq X.$$

For each cluster C_j, we compute its centroid

$$c_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i,$$

which represents the average position of all points within the cluster. The centroid serves as a compact summary of the cluster's geometry and becomes the representative node for subsequent hierarchical operations. The use of centroids as summaries aligns with practices in both classical clustering and modern point cloud analysis. For example, k-means clustering defines clusters precisely in terms of centroids, and hierarchical methods such as agglomerative clustering repeatedly merge based on centroid proximity. In point cloud learning, centroids are often employed in sampling strategies, such as the farthest point sampling used in PointNet++ to initialize local neighborhoods. By adopting centroids here, we unify a familiar abstraction from prior work with the density-driven clustering mechanism of DBSCAN. Importantly, however, unlike k-means, our centroids do not dictate cluster membership; rather, they summarize clusters that have already been discovered by density connectivity.

3.3 Relative Encoding of Points

Once centroids are established, the key challenge is how to represent the relationship between each point and its corresponding cluster center. Instead of preserving absolute coordinates, we define the relative encoding of a point $x_i \in C_j$ as

$$r_{i,j} = \big(d_{i,j}, u_{i,j}\big),$$

where

$$d_{i,j} = \|x_i - c_j\|_2, \quad u_{i,j} = \frac{x_i - c_j}{\|x_i - c_j\|_2}.$$

Here, $d_{i,j}$ is the Euclidean distance from the point to its centroid, and $u_{i,j}$ is the unit direction vector pointing from the centroid to the point. This design choice reflects several considerations. First, absolute coordinates are sensitive to global translation and scaling, which complicates generalization across datasets. By encoding points in terms of direction and distance relative to their centroid, we achieve a representation that is more robust to such global transformations. Second, this representation highlights the intrinsic structure of the cluster: distances capture radial distribution, while direction vectors capture angular spread. Prior research in vector quantization, spherical coding, and directional statistics has demonstrated the benefits of separating magnitude and orientation, and our method adapts this principle to point set representation. Moreover, relative encoding reduces redundancy in the representation. Instead of storing every raw coordinate, the cluster can be reconstructed from its centroid plus a set of relative encodings. This not only reduces memory requirements but also establishes a natural alignment across clusters and scales, since all encodings are defined relative to cluster-specific reference points.

3.4 Recursive Construction of Hierarchies

Once all centroids $\{c_1, \ldots, c_K\}$ are obtained, they form a reduced point set

$$X' = \{c_1, c_2, \ldots, c_K\}.$$

We then apply DBSCAN recursively to X', generating higher-level clusters and their centroids. Formally, let

$$X^{(0)} = X, \quad X^{(1)} = \{c_1^{(0)}, \ldots, c_{K_0}^{(0)}\}, \quad X^{(2)} = \{c_1^{(1)}, \ldots, c_{K_1}^{(1)}\}, \ldots$$

where $X^{(\ell)}$ is the set of centroids at level ℓ, K_ℓ is the number of clusters at that level, and $c_j^{(\ell)}$ denotes the j-th centroid. The recursion continues until DBSCAN returns a trivial partition, at which point no further meaningful clusters exist. This recursive procedure naturally induces a hierarchical structure in the form of a tree or forest. Each node corresponds to a centroid, and each edge links a centroid to the points or lower-level centroids it represents. The depth of the tree reflects the density complexity of the data: dense regions generate deeper branches with more detailed representations, while sparse regions terminate earlier. This adaptive hierarchy contrasts with fixed-depth octree structures, which partition space uniformly regardless of data density, and therefore may allocate resources inefficiently.

3.5 FINAL REPRESENTATION

The complete hierarchical representation is defined as

$$\mathcal{H} = \left\{ (c_j^{(\ell)}, R_j^{(\ell)}) \,\middle|\, \ell = 0, 1, \ldots, L,\ j = 1, \ldots, K_\ell \right\},$$

where $R_j^{(\ell)}$ denotes the set of relative encodings associated with centroid $c_j^{(\ell)}$, and L is the maximum depth of the hierarchy. This structure balances compactness and fidelity: it stores only a limited number of centroids at each level while preserving the local geometry of clusters through relative encoding. Compared to prior representations based on raw coordinates, fixed-radius neighborhoods, or axis-aligned partitions, our approach integrates density-awareness, recursive summarization, and relative geometric encoding into a unified framework. To consolidate the above components into a unified framework, we present the overall procedure in Algorithm 3.5. This pseudocode summarizes the step-by-step operations of our method, highlighting the recursive application of DBSCAN and the construction of hierarchical relative encodings. [t] Hierarchical DBSCAN-Based Point Set Representation Point set $X \subset \mathbb{R}^d$, DBSCAN parameters ε, MinPts Hierarchical representation $\mathcal{H}$ Initialize hierarchy $\mathcal{H} \leftarrow \emptyset$; Initialize level counter $\ell \leftarrow 0$; Set $X^{(0)} \leftarrow X$; $X^{(\ell)} \neq \emptyset$ Apply DBSCAN on $X^{(\ell)}$, obtaining clusters $C^{(\ell)}1, \ldots, C^{(\ell)}K_\ell$; cluster $C_j^{(\ell)}$ Compute centroid $c_j^{(\ell)} = \frac{1}{|C^{(\ell)}j|} \sum x \in C_j^{(\ell)} x$; For each $x \in C^{(\ell)}j$, compute relative encoding $ri, j = \left(|x - c_j^{(\ell)}|_2, \frac{x - c_j^{(\ell)}}{|x - c_j^{(\ell)}|_2}\right)$; Store $(c_j^{(\ell)}, R^{(\ell)}j)$ in $\mathcal{H}$; Set $X^{(\ell+1)} \leftarrow c^{(\ell)}1, \ldots, c^{(\ell)}K\ell$; Increment $\ell \leftarrow \ell + 1$; $\mathcal{H}$; Algorithm 3.5 illustrates the complete pipeline of our approach. At each level ℓ, the input point set $X^{(\ell)}$ is clustered using DBSCAN. Each cluster is summarized by its centroid, and the local geometry is encoded through relative distances and directions to that centroid. The set of centroids then serves as the input for the next level, ensuring that higher levels of the hierarchy capture progressively coarser structural summaries of the data. The procedure terminates once DBSCAN no longer produces nontrivial clusters, yielding a final hierarchical representation $\mathcal{H}$ that balances compactness and descriptive power.

3.6 RATIONALE FOR METHOD DESIGN

The design of our hierarchical framework is motivated by the need to address limitations in existing point set representations, such as sensitivity to global transformations, inefficiency in handling variable densities, and lack of adaptability to intrinsic data structures. By integrating DBSCAN for partitioning, we ensure that clusters reflect natural density distributions rather than imposed geometric assumptions, as seen in methods like k-means or octrees. The choice of centroid computation provides a stable summary point that facilitates recursive abstraction, drawing from established practices in clustering literature. Relative encoding further enhances robustness by decoupling local geometry from absolute positioning, enabling better generalization across scales and translations. This separation of distance and direction aligns with principles from directional statistics and vector quantization, promoting efficient reconstruction and reduced redundancy. The recursive hierarchy adapts to data complexity, allocating more levels to dense regions for finer detail while summarizing sparse areas compactly. Overall, these components form a cohesive system that prioritizes data-driven adaptability, computational efficiency, and representational fidelity, setting it apart from rigid or non-hierarchical alternatives.

3.7 COMPLEXITY ANALYSIS

The computational complexity of the proposed framework is dominated by the repeated application of DBSCAN and the centroid computation. For a point set of size n in d-dimensional space, the complexity of a single DBSCAN pass is $O(n \log n)$ when implemented with spatial indexing structures such as k-d trees or ball trees, and $O(n^2)$ in the worst case without indexing. The centroid computation for each cluster requires $O(|C_j| \cdot d)$ operations, and the relative encoding step adds an additional $O(|C_j| \cdot d)$ cost. Since each point belongs to exactly one cluster at each level, the total cost of centroid and encoding operations per level is $O(n \cdot d)$. Let L denote the number of recursive levels. In practice, L is logarithmic with respect to the dataset size, since each level reduces the number of representative points. The overall complexity of the hierarchical construction is therefore approximately

$$O\big(L \cdot (n \log n + n \cdot d)\big),$$

with the practical bound depending on the density distribution of the dataset. Storage complexity follows a similar trend: at each level, only centroids and relative encodings need to be stored, leading to a memory requirement of $O(n \cdot d + L \cdot k)$, where k is the number of centroids across all levels. This is significantly more efficient than storing all raw coordinates at full resolution. To contextualize the efficiency of our method, we compare its complexity to several widely used alternatives in Table 1. While k-means provides a lower per-iteration complexity, it requires a predefined number of clusters and often converges slowly. Octree-based methods achieve logarithmic query complexity but enforce a uniform partition that fails to adapt to varying densities. In contrast, our framework combines the adaptability of DBSCAN with hierarchical abstraction, achieving efficiency without sacrificing representational flexibility. The table highlights the key trade-offs. Our framework exhibits higher asymptotic complexity than k-means in individual iterations, but avoids the need for repeated iterations until convergence and adapts naturally to heterogeneous structures. Compared to octrees, our method incurs comparable computational cost while offering significantly better adaptability, since dense regions are represented with deeper hierarchical resolution while sparse regions are summarized more compactly. Thus, the proposed approach achieves a balanced compromise between efficiency, storage cost, and flexibility, making it well-suited for large-scale point set analysis where both global and local structures must be retained. In summary, the proposed hierarchical framework achieves a favorable trade-off between computational efficiency and representational fidelity. Its ability to adaptively capture density variations while maintaining a compact multi-scale structure distinguishes it from conventional clustering and partitioning methods. This balance forms the foundation for the empirical evaluation presented in the following section, where we demonstrate the effectiveness of the approach across diverse datasets and tasks.

Table 1: Complexity comparison of different point set representation methods.

Method	**Clustering Complexity**	**Storage Requirement**	**Adaptability to Density**
k-means	$O(nkd)$ per iteration	$O(n \cdot d)$	Low (requires fixed k)
Octree partitioning	$O(n \log n)$	$O(n)$	Low (uniform partition)
Our framework	$O(L \cdot (n \log n + n \cdot d))$	$O(n \cdot d + L \cdot k)$	High (density-aware)

4 Experiments

In this section, we present the experimental evaluation of the proposed hierarchical density-based abstraction framework. The primary objectives of the experiments are to (i) validate the feasibility of the method across different dimensionalities, (ii) analyze the scalability of the hierarchical construction process, and (iii) provide visualization of the resulting hierarchical structures in low-dimensional spaces. To achieve this, we design a set of synthetic point set datasets with controlled structures, and apply our method using a recursive DBSCAN-based clustering procedure.

4.1 Dataset Generation

Since real-world point sets vary widely in distribution, we generate synthetic datasets that allow us to carefully control the density and cluster structure. The datasets are designed to contain $250{,}000$ points distributed across multiple dimensions. To avoid a completely random distribution, which would hinder meaningful clustering, we explicitly arrange the data into 20 distinct clusters in each dataset. The cluster centers are separated by a fixed offset of 10 units along each dimension, ensuring that clusters remain well separated and can be identified without ambiguity. Formally, for a d-dimensional dataset, the i-th cluster center is positioned at

$$\mu_i = (10 \cdot i, 10 \cdot i, \ldots, 10 \cdot i) \in \mathbb{R}^d,$$

and each cluster is populated with approximately $\frac{250{,}000}{20} = 12{,}500$ points, with the exact number adjusted to evenly distribute the total. Within each cluster, Gaussian noise with zero mean and unit variance is added to simulate natural intra-cluster variation. This process ensures that while clusters remain separated, points within a cluster exhibit realistic spread rather than collapsing at the centroid. Datasets are generated for dimensions $d \in 1, 2, 3, 4$, resulting in four distinct point sets. Higher dimensions are also supported by the methodology but are excluded from visualization for clarity. The generation procedure is implemented in a Python script that outputs each dataset as a CSV file, with appropriate column naming conventions (e.g., $x_1, x_2, \ldots, x_d$).

4.2 EXPERIMENTAL PROCEDURE

The experimental procedure was carried out in two main stages: clustering and centroid computation. For each dataset dimension, the following steps were performed:

1. **Loading the dataset:** The generated CSV files containing the point sets were loaded into the clustering pipeline.
2. **Parameter selection:** For each level of clustering, an appropriate ε parameter for DBSCAN was determined using the k-distance graph method. The number of minimum points (`minPts`) was set as $d + 1$, where d denotes the data dimensionality.
3. **DBSCAN execution:** DBSCAN was applied to identify dense clusters. Noise points labeled as outliers were discarded.
4. **Centroid computation:** For each cluster, the centroid was calculated as the mean of all non-noise points within the cluster. The resulting centroids were then used as input for the next hierarchical level.
5. **Iteration and stopping criteria:** The procedure was repeated recursively until one of two conditions was met: (i) only a single centroid remained, or (ii) the centroids did not change across consecutive levels.
6. **Visualization:** For 1D, 2D, and 3D datasets, intermediate and final centroids were visualized to qualitatively illustrate the hierarchical clustering process.

This recursive framework yielded a multi-level representation of clusters, progressively condensing the data into higher-order centroids. The entire experimental workflow is automated through two scripts. The first script, $\texttt{generate}_d atasets.py, produces synthetic datasets in specified dimensions and saves them as CSV files. The second scr$

4.3 EXPERIMENTAL RESULTS

We summarize the hierarchical clustering results for datasets of 1D through 4D. Each dataset initially consisted of 20 clusters at Level 0. Across levels, the number of centroids decreased, illustrating the merging process until a single global centroid was reached.

One-dimensional case. In the 1D dataset, the initial 20 clusters were progressively merged into 6 centroids at Level 1, then reduced to 3 centroids at Level 2, and finally collapsed into a single centroid at Level 3. This demonstrates the algorithm's ability to capture fine-grained structure at the lower levels while converging to a global representation at higher levels.

Two-dimensional case. For the 2D dataset, the initial 20 clusters were immediately merged into a single centroid at Level 1, located at approximately $(95.0, 95.0)$. This rapid condensation indicates a higher degree of symmetry in the spatial distribution of clusters, where the global centroid is stable and representative of the dataset as a whole.

Three-dimensional case. The 3D dataset followed a similar trend to the 2D case. Starting from 20 clusters at Level 0, the algorithm converged in the next iteration to a single centroid around $(95.0, 95.0, 95.0)$. This outcome demonstrates that in higher-dimensional spaces, the recursive DBSCAN procedure still effectively condenses distributed clusters into a global centroid.

Four-dimensional case. In the 4D dataset, the hierarchical process also converged from 20 initial clusters to a single centroid at Level 1, located at approximately $(95.0, 95.0, 95.0, 95.0)$. This consistent behavior across 2D–4D highlights the robustness of the proposed method in handling multi-dimensional data distributions.

4.4 VISUALIZATIONS

The following figure presents visualizations produced by the pipeline for the 1D, 2D, and 3D datasets. Each subfigure overlays centroids at different hierarchical levels on top of the original point distribution to illustrate progressive abstraction. The images corroborate the numerical summary in

Table 2: Summary of hierarchical clustering results across dimensions.

Dimension	**Levels**	**Centroids per Level**	**Final Centroid (approx.)**
1D	4	$20 \rightarrow 6 \rightarrow 3 \rightarrow 1$	71.3
2D	2	$20 \rightarrow 1$	(95.0, 95.0)
3D	2	$20 \rightarrow 1$	(95.0, 95.0, 95.0)
4D	2	$20 \rightarrow 1$	(95.0, 95.0, 95.0, 95.0)

0.32 [width=]points$_1 d_v isualization.png$

Figure 1: 1D hierarchical partitioning

0.32 [width=]points$_2 d_v isualization.png$

Figure 2: 2D hierarchical partitioning

0.32 [width=]points$_3 d_v isualization.png$

Figure 3: 3D hierarchical partitioning

Figure 4: Hierarchical decomposition of sampled points in 1D, 2D, and 3D spaces. Each subfigure illustrates the progressive subdivision of the search space, with points grouped at different levels of granularity.

Table 2 and provide additional qualitative insights. Figure 4(a) (1D) clearly shows the original cluster centers aligned along the number line and the staged reduction of centroids across levels; the intermediate markers (Level 1 and Level 2) illustrate how neighboring clusters are aggregated into larger groups before a single global centroid is reached. Figures 4(b) and 4(c) show that, in two and three dimensions, centroids at Level 0 lie tightly on the diagonally distributed cluster centers, and the Level 1 centroid lies near the geometric center of those clusters. Visually, the immediate coalescence into a single Level 1 centroid for 2D and 3D matches the previously reported numeric results. To further validate the efficiency of the hierarchical clustering strategy, we performed a detailed quantitative analysis across different dimensional settings (1D to 4D). Table 3 summarizes the number of representative points identified at each level of decomposition. The results highlight

Dimension	Level 0	Level 1	Level 2	Level 3
1D	20	6	3	1
2D	20	1	–	–
3D	20	1	–	–
4D	20	1	–	–

Table 3: Number of representative points at different hierarchical levels across dimensions. "–" indicates no further subdivision was performed.

a consistent trend: the initial level (Level 0) always contains the densest set of representative points, distributed at approximately uniform intervals across the search space. For the 1D case, the decomposition proceeds through four levels, gradually reducing the number of representative points from 20 at Level 0 to a single central point at Level 3. This indicates a highly efficient compression of the search space while retaining structural balance. In higher dimensions (2D, 3D, and 4D), the decomposition becomes shallower, typically converging within one or two levels. For example, in the 2D case, 20 representative points at Level 0 condense into a single representative point at Level 1, suggesting that higher-dimensional structures stabilize faster under the hierarchical partitioning. A similar phenomenon is observed in the 3D and 4D cases. This dimensional sensitivity implies that while finer granularity is needed in 1D to capture local structures, higher-dimensional domains benefit from the increased redundancy of spatial information, which accelerates convergence. Overall, these findings quantitatively demonstrate that the hierarchical decomposition scales effectively across dimensions. It balances exploration (dense sampling at Level 0) and exploitation (progressive reduction of representative points), enabling both efficient coverage and computational tractability.

4.5 DISCUSSION

The experiments confirm several important aspects of the proposed framework. First, the hierarchical construction consistently terminates in a small number of levels, typically between 3 and 6, due to the rapid reduction in the number of centroids per level. This validates the logarithmic scaling behavior hypothesized in the complexity analysis. Second, the use of k-distance graphs at each level enables adaptive selection of ε, which is crucial for maintaining clustering quality across different abstraction levels. Third, visualization in 1D to 3D clearly illustrates the hierarchical nature of the framework, with clusters progressively summarized into centroids while retaining the underlying global structure. The experimental results reveal several important insights. First, the hierarchical clustering framework reliably condensed structured multi-cluster datasets into progressively fewer centroids, ultimately converging to a single representative centroid. Second, while the 1D dataset required multiple levels of merging, the 2D–4D datasets exhibited faster convergence, often collapsing directly into a global centroid after the first iteration. This difference suggests that dimensionality and the relative geometry of cluster placement (e.g., linear spacing versus multidimensional spread) significantly influence the hierarchical depth required for convergence. Finally, the consistency of centroid positions across different dimensions—located near the global mean of cluster centers—validates the effectiveness of the method in capturing underlying structural regularities. The image-based analysis provides additional interpretive value. The 1D visualization (Figure 4(a)) makes it apparent that neighboring clusters tend to merge sequentially in a chain-like fashion along the line, producing several intermediate abstraction levels. In contrast, the 2D and 3D visualizations (Figures 4(b) and 4(c)) reflect a configuration in which cluster centroids are arranged symmetrically around a central region; under the automatically selected DBSCAN parameters (via the k-distance heuristic), those centroids are dense enough relative to the chosen ε to be identified as a single higher-level cluster. In practical terms, this observation highlights the sensitivity of the hierarchical depth to DBSCAN's parameterization: larger ε (or smaller `minPts`) at the centroid levels will promote faster merging, whereas more conservative choices will tend to preserve intermediate levels. Finally, these results suggest practical guidelines for using the method in applied settings. If the objective is to obtain multi-level decompositions with richer intermediate representations, practitioners should consider tuning ε and `minPts` at each level (or using level-dependent heuristics) to avoid premature collapse. Conversely, if aggressive condensation is desired for compression or coarse-grained summarization, the current heuristic (k-distance selection and `minPts` = $d+1$) provides a straightforward and effective strategy. Overall, the experimental evaluation demonstrates that the proposed method is both scalable and interpretable. By leveraging density-aware clustering and recursive abstraction, it achieves a compact representation of large point sets without sacrificing fidelity. This establishes a strong foundation for applying the framework to more complex real-world datasets, which will be explored in future work.

5 CONCLUSION

This study introduces a novel hierarchical framework for point set representation that integrates density-based clustering with relative geometric encoding. By leveraging DBSCAN to partition point sets and recursively constructing a tree-like structure of centroids and relative encodings, our approach addresses key limitations in existing methods, including sensitivity to sampling density, reliance on fixed neighborhood parameters, and lack of adaptability to intrinsic data distributions. The proposed method achieves a compact yet expressive representation that balances computational efficiency with fidelity to local and global geometric structures.

Experimental results across synthetic datasets in 1D to 4D demonstrate the framework's scalability and robustness. The hierarchical construction consistently converges within a small number of levels, with the depth of the hierarchy reflecting the density complexity of the data. Visualizations and quantitative analyses confirm that the method effectively captures multi-scale structures, with centroids summarizing clusters at varying levels of granularity. The use of adaptive parameter selection via k-distance graphs further enhances the framework's ability to handle heterogeneous and noisy point sets without imposing rigid structural assumptions.

Compared to prior approaches, such as PointNet++, DGCNN, and octree-based methods, our framework offers improved adaptability to density variations and reduced sensitivity to global transformations through relative encoding. While transformer-based models and pretraining strategies have

shown promise, their computational complexity limits scalability for large point sets. In contrast, our method achieves logarithmic scaling with respect to dataset size, as validated by complexity analysis, making it well-suited for large-scale 3D analysis.

Future work will extend this framework to real-world datasets, exploring its applicability to tasks such as semantic segmentation and object detection in 3D point clouds. Additionally, integrating learning-based components, such as neural networks for refining cluster assignments or optimizing relative encodings, could further enhance performance. Overall, this study establishes a strong foundation for density-aware, hierarchical point set representations, offering a versatile and efficient solution for processing complex, high-dimensional point data.

ACKNOWLEDGMENTS

We thank Grok4 for assisting in searching related works and GPT-5 for polishing the article.

REFERENCES